

Laptop Price Prediction

Team Members:

Team Leader: Karim Amr Abdelmonem

Second Member: Malak Mowaffak Shawky

Third Member: Omar Ali Hussien

Fourth Member: Youssef Albir Soliman

Fifth Member: Kirolous Ashraf Raouf

Member	Task
Omar Ali Hussien	Data Preprocessing and Analysis
Malak Mowaffak Shawky	
Kirolous Ashraf Raouf	
Karim Amr Abdelmonem	Modleing Validation and Tuning
Youssef Albir Soliman	

Contents

1.0 Data Description and Model Purpose:	3
1.1 Key results	3
1.2 Conclusion	3
2.0 Data Exploration & Analysis (EDA)	4
Data Overview, Statistical Summary & Missing Value Analysis	4
Correlation Analysis.....	5
Central Tendency	5
3.0 Data Cleaning and Feature Engineering	6
One Hot Encoding.....	6
Structural transformation	6
4.0 Feature Selection and Dataset Preparation	7
5.0 Modeling Validation and Tuning.....	9
Learning Curve Analysis.....	10
Feature Scaling	11
Polynomial Regression (Degree 2)	11
Regularization (Ridge and Lasso)	11
Ridge Regression	11
Lasso Regression.....	12
Decision Tree Regressor.....	12
Multi-Model Benchmarking with Pipelines	12
6.0 Model Evaluation and Comparison	13
Evaluation Metrics	13
Observations	14
7.0 Hyperparameter Tuning.....	14
Ridge Regression Tuning	14
Polynomial Regression Tuning	14
Decision Tree Tuning	15
8.0 Final Model Selection	15

Laptop Prediction

1.0 Data Description and Model Purpose:

Our dataset contains detailed specifications for different laptops along with their market prices. It helps us understand how the technical features of a laptop relate to its real-world value. The dataset is divided into two parts: a training set of 881 laptops, where both the specifications and prices are known, and a testing set of 422 laptops used to evaluate how well the model performs.

Each laptop includes a wide range of hardware specifications such as CPU frequency, RAM size, and storage capacity. These are the kinds of features people usually look at when comparing laptops online or in stores.

In addition to internal hardware, the dataset also includes physical characteristics like screen size, screen resolution, and weight. Brand information is also included and categorized so the model can recognize differences between manufacturers.

The main objective of this project is to build a regression model that can accurately estimate the market price of a laptop based on these specifications.

By analyzing how different technical features influence price, the model helps highlight the key factors that determine laptop value—such as RAM size and processor speed. This allows us to create a data-driven approach to estimating laptop prices and understanding pricing patterns within the electronics market.

1.1 Key results

The final regression model demonstrated strong predictive performance when estimating laptop prices based on their specifications. During evaluation, the model achieved an R^2 value of approximately **0.81**, which means it can explain about **81% of the variation in laptop prices** within the dataset.

In addition, the Mean Absolute Error (MAE) remained within an acceptable range. This indicates that, on average, the difference between the predicted price and the actual market price is relatively small.

Overall, these results show that the model performs reliably and is capable of providing realistic price estimates for a wide range of laptop configurations.

1.2 Conclusion

The analysis shows that certain technical specifications play a major role in determining laptop prices. In particular, **RAM capacity** and **CPU speed** were found to have a strong positive relationship with price. As these specifications increase, laptop prices tend to rise as well.

Laptop category also plays an important role. Devices classified as **Gaming laptops** or **Ultrabooks** tend to be more expensive because they usually include specialized hardware such as powerful graphics cards and high-performance processors.

On the other hand, features like **screen size** were found to have a weaker relationship with price. This suggests that buyers and manufacturers place greater value on internal performance components rather than just physical dimensions.

Overall, the results confirm that laptop pricing is driven mainly by internal hardware performance and storage technology rather than purely external features.

2.0 Data Exploration & Analysis (EDA)

Data Overview, Statistical Summary & Missing Value Analysis

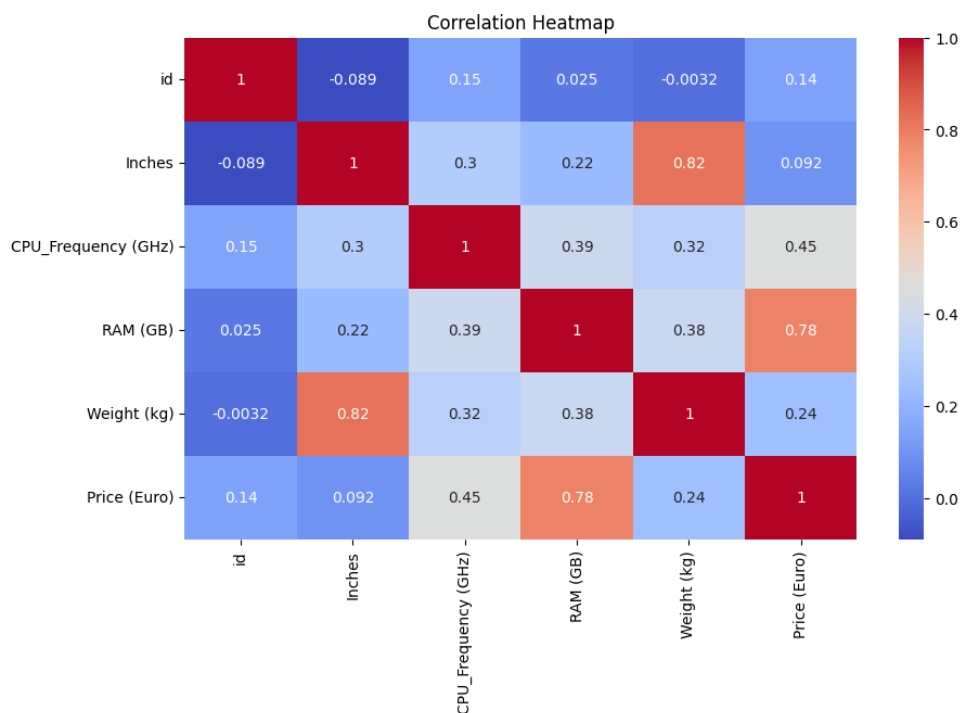
The first step in our analysis was to explore the structure of the dataset. This involved checking the number of records, identifying the types of data in each column, and reviewing the overall statistical distribution of numerical variables.

By performing this initial inspection, we were able to understand the general range of laptop prices and hardware specifications within the training dataset.

Another important step was checking for missing values. We calculated the number of null entries in each feature to determine whether any data cleaning or imputation would be required before training our models. Ensuring that the dataset is complete and consistent is essential for building reliable machine learning models.

```
... Missing values in train:
id                0
Company           0
Product           0
TypeName          0
Inches            0
ScreenResolution  0
CPU_Company       0
CPU_Type          0
CPU_Frequency (GHz) 0
RAM (GB)          0
Memory           0
GPU_Company       0
GPU_Type          0
OpSys             0
Weight (kg)       0
Price (Euro)      0
dtype: int64
```

Correlation Analysis

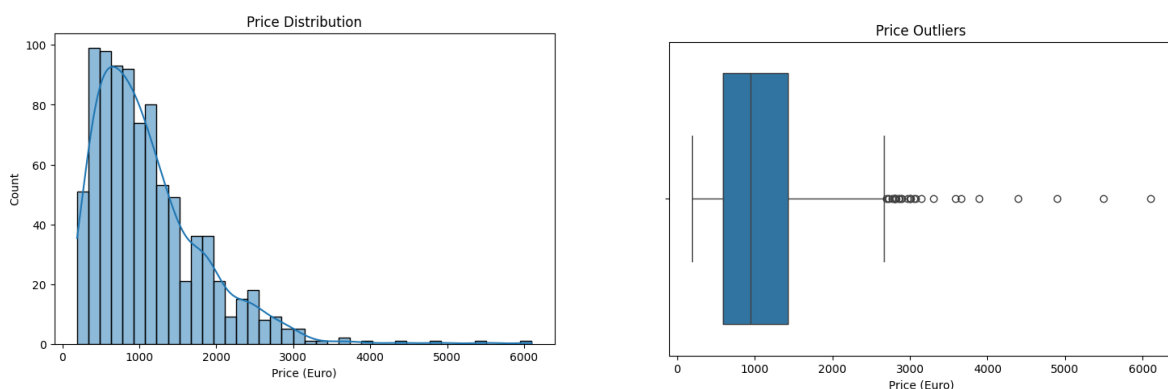


To better understand the relationships between different numerical variables, we generated a **correlation heatmap**. This visualization highlights how strongly different features are related to each other and to the target variable (price).

The heatmap revealed several important insights:

- **RAM** showed the strongest relationship with price, with a correlation coefficient of **0.74**, indicating that laptops with more RAM are generally more expensive.
- A strong correlation of **0.82** was observed between **screen size (inches)** and **weight**, which is expected since larger laptops tend to weigh more.

This type of analysis helps identify which features are most useful for predicting price while also revealing cases where multiple variables provide very similar information.



The **price histogram** showed a right-skewed distribution. This means most laptops in the dataset fall within the **budget to mid-range price category**, while only a smaller number of laptops belong to the high-end premium segment.

A **boxplot visualization** also revealed several outliers above **€2,500**. These likely represent premium or high-performance laptops that sit far outside the typical price range.

Central Tendency

Looking at the distribution of prices, the **median price** appears relatively low compared to the highest values in the dataset. This confirms that while a few expensive laptops exist, the majority of devices are priced in the lower to mid-range market segment.

3.0 Data Cleaning and Feature Engineering

Before applying any machine learning models, we prepared and cleaned the dataset to ensure it could be processed effectively.

To begin, we created a **deep copy of the dataset** so that all transformations could be performed without modifying the original data.

We then focused on handling missing values and ensuring that all numerical features were formatted consistently so the models could interpret them correctly.

One Hot Encoding

Several variables in the dataset were categorical (text-based), such as laptop brand or operating system. Machine learning models cannot directly process text values, so we converted these categories into a numerical format using **One-Hot Encoding**.

This process creates a separate binary column for each category. If a laptop belongs to that category, the value is **1**; otherwise it is **0**.

One-Hot Encoding was applied to the following features:

- Company Name
- Laptop Type
- Operating System
- GPU Company
- CPU Company
- Memory Type

After the new encoded columns were created, the original text columns were removed because they were no longer needed.

Structural transformation

Some features required additional restructuring so that they could be interpreted numerically by the model.

Screen Resolution Splitting

The original screen resolution column contained text values (for example: 1920x1080). We split this into two separate numerical columns:

- Screen Width
- Screen Height

This allows the model to analyze screen resolution directly as pixel counts.

Memory Conversion

Storage values originally appeared as text (for example: 512GB or 1TB). These values were converted into a consistent numerical format measured in **gigabytes**. This ensures that the model understands the true scale of storage capacity rather than treating the values as simple labels.

```
data.columns

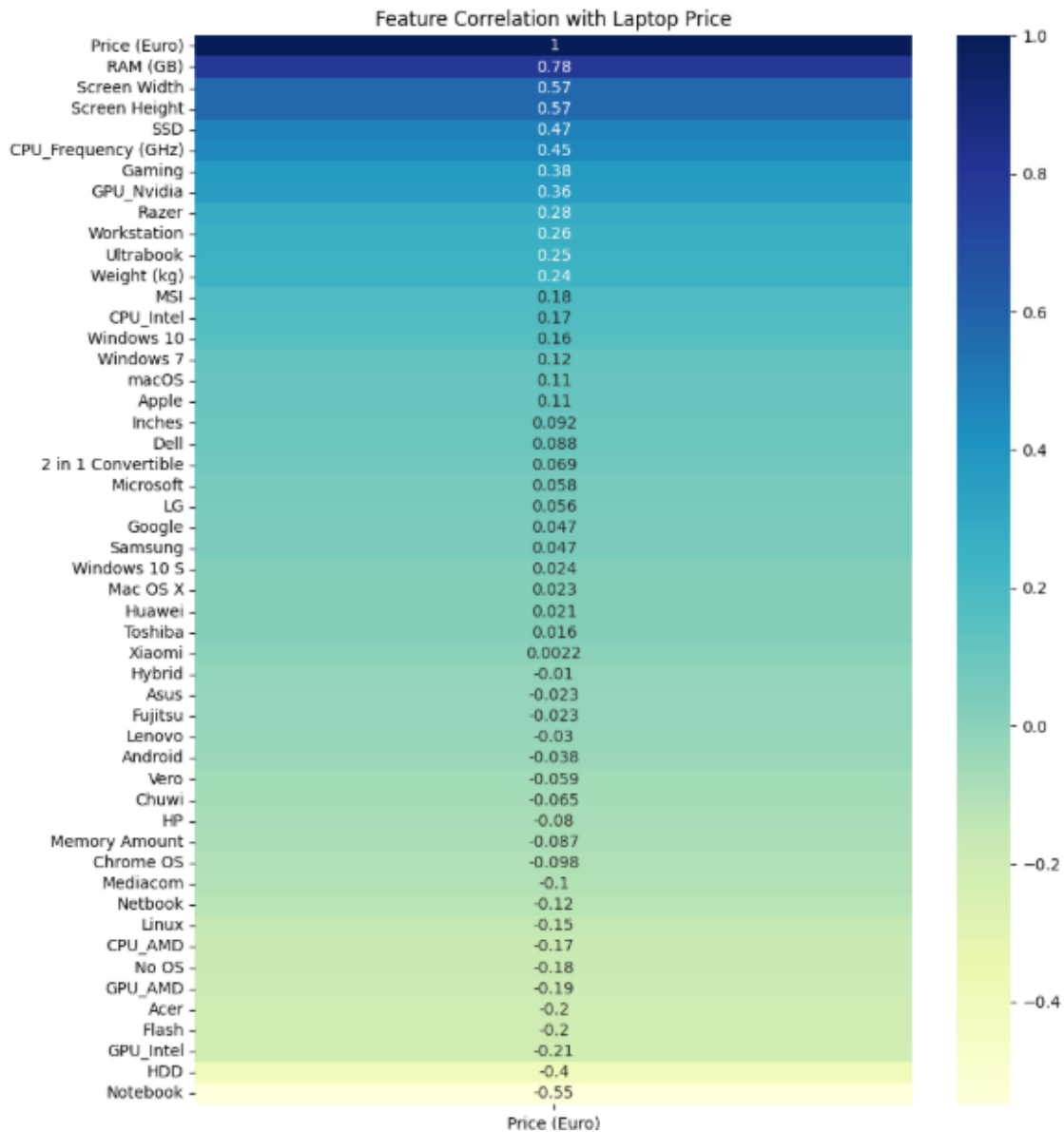
Index(['Inches', 'CPU_Frequency (GHz)', 'RAM (GB)', 'Weight (kg)',
      'Price (Euro)', 'Acer', 'Apple', 'Asus', 'Chuwi', 'Dell', 'Fujitsu',
      'Google', 'HP', 'Huawei', 'LG', 'Lenovo', 'MSI', 'Mediacom',
      'Microsoft', 'Razer', 'Samsung', 'Toshiba', 'Vero', 'Xiaomi',
      '2 in 1 Convertible', 'Gaming', 'Netbook', 'Notebook', 'Ultrabook',
      'Workstation', 'Android', 'Chrome OS', 'Linux', 'Mac OS X', 'No OS',
      'Windows 10', 'Windows 10 S', 'Windows 7', 'macOS', 'Screen Width',
      'Screen Height', 'GPU_AMD', 'GPU_Intel', 'GPU_Nvidia', 'CPU_AMD',
      'CPU_Intel', 'Memory Amount', 'Flash', 'HDD', 'Hybrid', 'SSD'],
      dtype='object')
```

4.0 Feature Selection and Dataset Preparation

To determine which variables were most useful for predicting laptop prices, we performed feature correlation analysis.

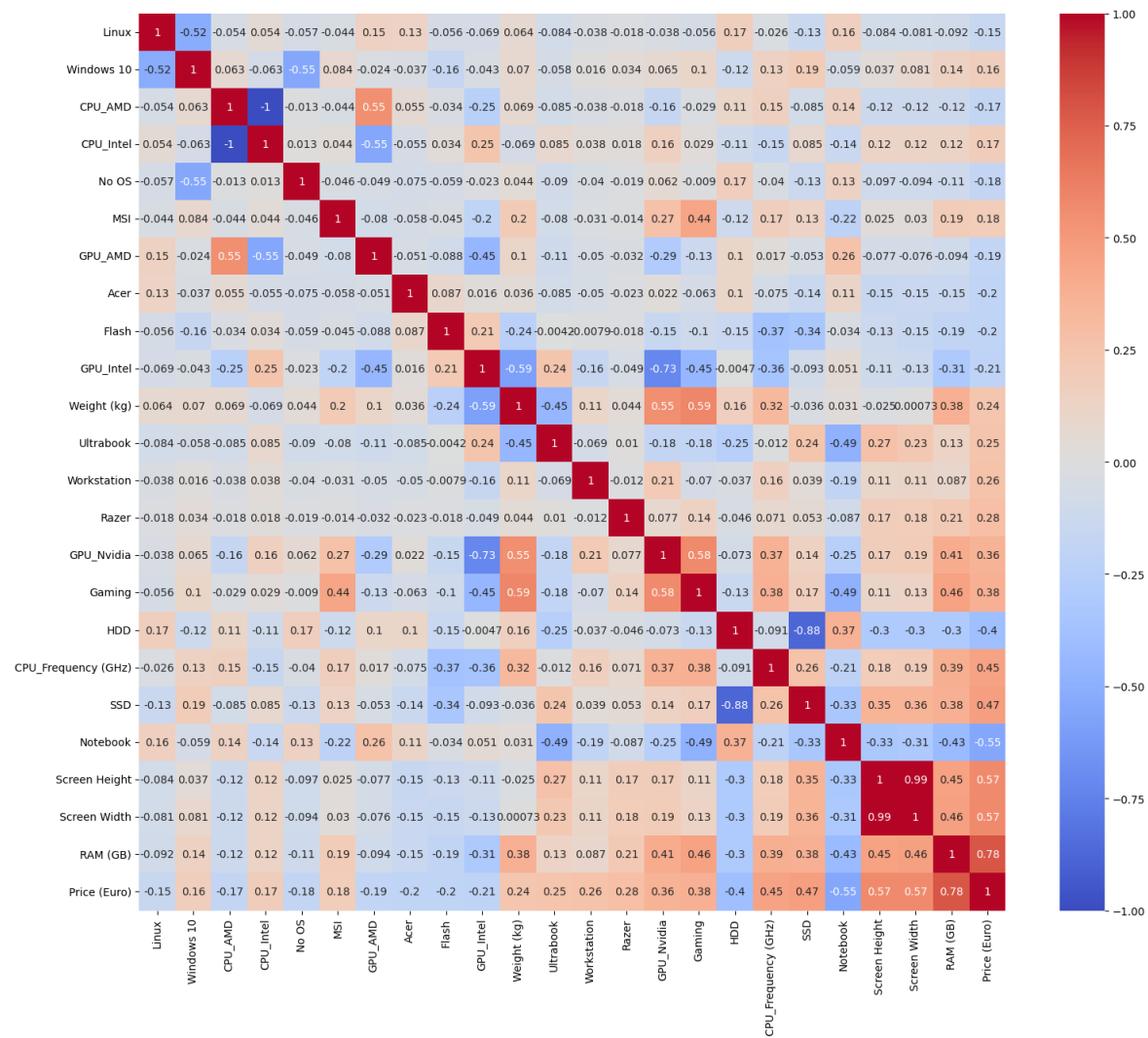
The results showed several key relationships. For example, **CPU Frequency (GHz)** had a moderate positive correlation of **0.43** with price, suggesting that laptops with faster processors tend to be more expensive.

In contrast, **screen size (inches)** showed almost no correlation with price (approximately **0.068**). This indicates that larger screens do not necessarily mean higher prices, and that internal hardware specifications have a much stronger influence on cost.



Based on this analysis, we selected the **top 24 features** that had the strongest relationship with the target variable.

Selecting only the most relevant features helps improve model performance by removing unnecessary or redundant variables. This ensures the model focuses on the specifications that truly influence laptop prices.



5.0 Modeling Validation and Tuning

At this stage of the project, the focus shifts from preparing the data to evaluating and improving machine learning models.

The main goal is to ensure that the models can generalize well to new, unseen data rather than simply memorizing the training dataset.

To achieve this, we used **K-Fold Cross-Validation**.

In this method, the dataset is divided into several smaller subsets called **folds**. During each iteration:

- One fold is used as the validation set.
- The remaining folds are used to train the model.

This process repeats until each fold has been used as the validation set once. The final evaluation metrics are then averaged across all folds, producing a more reliable estimate of model performance.

This approach reduces the risk of overfitting and ensures that results are not dependent on a single random train-test split.

Learning Curve Analysis

After cross-validation, we generated a **learning curve** to analyze how the model's performance changes as the training dataset grows.

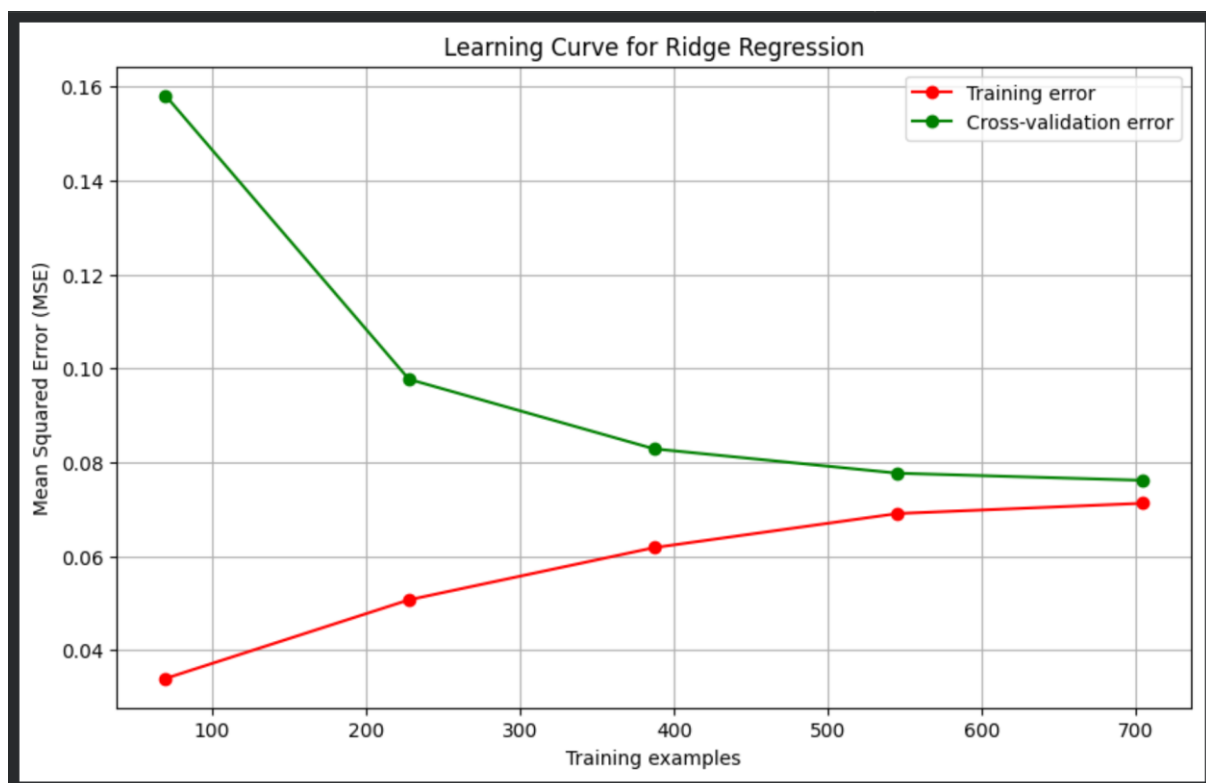
The learning curve compares two values:

- **Training Error**
- **Cross-Validation Error**

These curves help us understand how the model behaves:

- If training error is low but validation error is high, the model is **overfitting**.
- If both errors are high, the model is **underfitting**.
- If the curves gradually converge, the model has a good balance between **bias and variance**.

Bias represents errors caused by overly simple assumptions, while variance reflects how sensitive the model is to changes in the training data. Increasing model complexity usually reduces bias but increases variance.



Feature Scaling

Before training the models, we standardized the numerical features using **StandardScaler**.

Standardization transforms each feature so that:

- Mean = 0
- Standard Deviation = 1

Scaling is important because many machine learning algorithms are sensitive to the scale of input variables.

Without scaling, large-scale variables like storage capacity might dominate the learning process, while smaller-scale variables such as CPU frequency might have very little influence.

Standardization ensures that every feature contributes fairly during model training.

Polynomial Regression (Degree 2)

Polynomial Regression was introduced to capture **non-linear relationships** between laptop specifications and price.

Unlike standard linear regression, polynomial regression allows the model to include additional terms such as:

- Squared features
- Interactions between variables

We selected a **degree of 2**, which provides enough flexibility to capture more complex relationships without dramatically increasing model complexity.

Higher polynomial degrees were avoided because they can introduce a very large number of features, increasing computational cost and the risk of overfitting.

Regularization (Ridge and Lasso)

To prevent models from becoming overly complex, we applied **regularization techniques**.

Ridge Regression

Ridge Regression uses **L2 regularization**, which adds a penalty proportional to the square of model coefficients.

This has several benefits:

- Reduces the magnitude of coefficients
- Prevents the model from relying too heavily on a single feature
- Improves generalization to new data

The strength of this penalty is controlled by the **alpha (α)** parameter.

Lasso Regression

Lasso Regression applies **L1 regularization**, which penalizes the absolute value of coefficients.

A key advantage of Lasso is that it can shrink some coefficients all the way to **zero**, effectively removing less important features from the model.

Because of this property, Lasso can act as an **automatic feature selection method**.

Decision Tree Regressor

In addition to regression models, we also evaluated a **Decision Tree Regressor**, which can capture complex nonlinear relationships in the data.

Decision trees work by repeatedly splitting the dataset into smaller subsets based on feature values. Each split creates a decision rule, and the final predictions are generated at the leaf nodes of the tree.

Advantages of decision trees include:

- Ability to model nonlinear patterns
- No requirement for feature scaling
- High interpretability

However, trees can easily overfit if they grow too deep. To avoid this, parameters such as **maximum depth** and **minimum samples per split** were tuned.

Multi-Model Benchmarking with Pipelines

To ensure a fair comparison between models, we used **Scikit-Learn pipelines**.

Pipelines combine preprocessing steps and model training into a single workflow. This ensures that transformations like feature scaling and polynomial feature generation are applied consistently during both training and validation.

The models evaluated in this benchmarking stage were:

- Linear Regression

- Polynomial Regression
- Ridge Regression
- Lasso Regression
- Decision Tree Regressor

All models were evaluated using **5-Fold Cross-Validation** with the following metrics:

- Mean Absolute Error (MAE)
- Root Mean Squared Error (RMSE)
- R^2 Score

Using the same evaluation framework for every model ensures that performance comparisons are fair and reliable.

6.0 Model Evaluation and Comparison

After training the models, their performance was compared using multiple regression evaluation metrics.

Evaluation Metrics

R^2 Score

This metric measures how well the model explains the variation in laptop prices. Values closer to **1** indicate stronger predictive performance.

Mean Absolute Error (MAE)

MAE calculates the average difference between predicted prices and actual prices. Lower values indicate better accuracy.

Root Mean Squared Error (RMSE)

RMSE measures the square root of the average squared prediction errors. It penalizes larger prediction mistakes more strongly than MAE.

```
--- Cross-Validation Results ---
Model: Linear Regression
Mean R-squared: 0.7992 (+/- 0.0212)
Mean MAE: 0.2192 (log scale)
Mean RMSE: 0.2760 (log scale)

Model: Polynomial Regression (Deg 2)
Mean R-squared: 0.8346 (+/- 0.0186)
Mean MAE: 0.1955 (log scale)
Mean RMSE: 0.2504 (log scale)

Model: Ridge Regression
Mean R-squared: 0.7993 (+/- 0.0211)
Mean MAE: 0.2192 (log scale)
Mean RMSE: 0.2759 (log scale)

Model: Lasso Regression
Mean R-squared: -0.0052 (+/- 0.0064)
Mean MAE: 0.5060 (log scale)
Mean RMSE: 0.6197 (log scale)

Model: Decision Tree
Mean R-squared: 0.7570 (+/- 0.0112)
Mean MAE: 0.2240 (log scale)
Mean RMSE: 0.3045 (log scale)
```

Observations

From the evaluation results we observed that:

- Linear Regression provided a solid baseline model.
 - Polynomial Regression improved the ability to capture nonlinear relationships.
 - Ridge Regression improved generalization by controlling coefficient size.
 - Lasso Regression simplified the model by removing less important features.
 - Decision Trees captured nonlinear patterns but required careful tuning to avoid overfitting.
-

7.0 Hyperparameter Tuning

To further improve performance, we performed **hyperparameter tuning** using **GridSearchCV**.

GridSearchCV systematically tests different combinations of model parameters and identifies the configuration that produces the best cross-validation results.

Ridge Regression Tuning

Hyperparameter tuned:

- **Alpha (α)** – controls the strength of regularization.

```
--- Grid Search: Ridge Regression ---  
Fitting 5 folds for each of 6 candidates, totalling 30 fits  
Best Ridge Parameters: {'model__alpha': 10}  
Best Ridge CV Score: -0.07609847667269615
```

Polynomial Regression Tuning

Hyperparameter tuned:

- **Polynomial Degree**

Degrees tested:

- 1
- 2
- 3
- 4

```
--- Grid Search: Polynomial Regression ---  
Fitting 5 folds for each of 4 candidates, totalling 20 fits  
Best Polynomial Parameters: {'poly_features__degree': 2}  
Best Polynomial CV Score: -0.06272051296261519
```

Decision Tree Tuning

Hyperparameters tuned:

- **max_depth**
- **min_samples_split**
- **min_samples_leaf**

These parameters help control the complexity of the tree and reduce overfitting.

```
--- Grid Search: Decision Tree ---  
Fitting 5 folds for each of 54 candidates, totalling 270 fits  
Best Tree Parameters: {'model__max_depth': 15, 'model__min_samples_leaf': 4, 'model__min_samples_split': 10}  
Best Tree CV Score: -0.07486615458750849
```

8.0 Final Model Selection

After evaluating all models and tuning their parameters, the model with the best overall cross-validation performance was selected as the final model.

The selection was based on three main criteria:

- Highest **R² Score**
- Lowest **RMSE**
- Lowest **MAE**

We also ensured that the model demonstrated stable performance across different validation folds, confirming that it generalizes well to unseen data.

```
==== BEST MODELS SUMMARY ====  
  
Best Ridge Params: {'model__alpha': 10}  
Best Polynomial Params: {'poly_features__degree': 2}  
Best Decision Tree Params: {'model__max_depth': 15, 'model__min_samples_leaf': 4, 'model__min_samples_split': 10}  
  
Best Model Overall: Polynomial
```